

- **SXR — SCL eXecution Runtime (execução/distribuição do IR)**
- **Resumo (1 parágrafo)**

SXR é a camada que **embala e executa** o **IR** (intermediate representation) gerado por **SLang** (.slang, script procedural) e **SCL** (.scl, contrato declarativo com $\delta/\kappa/\lambda$). O **IR** é o “bytecode” comum. O SXR entrega três sabores: **SXR-Native** (binário/one-file), **SXR-JS** (Node/Deno/Browser) e **SXR-API** (serviço HTTP). A semântica fica no **IR**; SXR só roda/empacota com segurança e determinismo (sem eval aberto).

Pipeline canônico

.slang --(slangc.py)--> IR JSON --\

.scl --(sclc.py)----> IR JSON ----> SXR (Native | JS | API) => execução

Contrato estável do IR: "ir": "svm-ir/1".

Executor de referência: SVM (Python/TS) com run_ir(ir) / SVM.run().

Casos industriais (comandos e outputs)

1) Fintech — Reembolso com RBAC, redação PII e auditoria

Entrada (SCL, contracts/refund.scl):

1. [stm-context v2.6]

2. S/seq=validate->compute->decision

K/policy=if requester.role $\notin$ {finance,dpo} -> redact:[email,cpf]

- K/policy=if consent!=true -> deny:"missing_consent"
- L/refund_value=order_total*refund_pct
-
- D/1@A=proposal:order_id:"O-8831"|customer:"Ana"|email:"ana@ex.com"|
cpf:"123.456.789-00"|order_total:699.90|refund_pct:0.20|consent:true

-
- **Compilar → IR → rodar (SXR-Native/JS/API equivalem; exemplo SXR-API Python):**

-
- `python tools/lang/scl.py emit-ir contracts/refund.scl > builds/refund.ir.json`
- `python tools/sxr/api_py.py serve --port 8080`
- `curl -s -X POST :8080/run -H "content-type: application/json" -d @builds/refund.ir.json`
-
- **Saída (ex.: role=finance)**

```
{ "ok": true, "result": { "refund_value": 139.98, "email": "ana@ex.com", "cpf":  
"123.456.789-00" },  
"meta": { "engine": "SXR-API", "ir_ver": "svm-ir/1" } }
```

- - **Saída (role=support, PII redigida):**
- 1.
 2.

```
{ "ok": true, "result": { "refund_value": 139.98, "email": "■ REDACTED ■", "cpf":  
"■ REDACTED ■" } }
```
 - 3.

Sem consentimento:

```
{ "ok": false, "error": "missing_consent" }
```

Mostra RBAC, minimização de dados e trilha de auditoria (hash interno do executor).

2) Energia — Oferta de energia com limites operacionais e cálculos

Entrada (SCL, contracts/energy_trade.scl):

[stm-context v2.6]

S/seq=topic->context->proposal->check->answer

topic=EnergyTrade|context=floor-3

K/max_power_kW=2.5

K/if:battery<20->reject:"bateria baixa"

L/cost=price_kWh*qty_kWh

L/eta_h=qty_kWh/power_kW

D/1@A=proposal:qty_kWh:0.6|power_kW:0.25|price_kWh:0.55|battery:68

Run (SXR-Native CLI):

```
python tools/lang/sclc.py emit-ir contracts/energy_trade.scl > builds/energy.ir.json
tools/sxr/native_pack.py --ir builds/energy.ir.json # gera dist/energy/energy(.exe)
dist/energy/energy
```

Saída:

δ/1@assistant=answer:"cost=0.33; eta_h=2.4"

Troque battery:10 na linha D/1@A e o executor retorna:

δ/1@assistant=answer:"bateria baixa"

Demonstra **κ** precedendo **λ** (bloqueio antes de calcular), útil para **SCADA/DER** e **resposta à demanda**.

3) E-commerce — Pedido com validação de slots e subtotal

Entrada (SCL, contracts/order.scl):

[stm-context v2.6]

S/seq=topic->context->intent->slots->plan->answer

topic=OrderFlow

K/require=object

K/max_qty=10

K/ask_if_missing=qty:"Quantas unidades?"|price:"Qual preço unitário?"

L/subtotal=qty*price

D/7@user=utterance:"comprar pão"

D/7@parser=intent:Buy|object:pão|qty:?.|price:2.50

Run (SXR-JS via Deno, binário nativo):

```
python tools/lang/sclc.py emit-ir contracts/order.scl > builds/order.ir.json
deno compile --allow-read tools/sxr/js_runner.ts # gera ./js_runner(.exe)
./js_runner builds/order.ir.json
```

Saída (faltando qty):

```
δ/7@assistant=ask:"Quantas unidades?"
```

Depois, alimentando qty:2:

```
δ/7@assistant=answer:"subtotal=5.0"
```

Case padrão de **carrinho/checkout** com UX guided (ask) e **limites**.

4) Saúde — Prescrição com regra clínica e rastreamento

Entrada (SCL, contracts/rx_rule.scl):

```
[stm-context v2.6]
```

```
S/seq=validate->compute->decision
```

```
# Regra clínica (ilustrativa): contraindicação se GFR<30 para droga X
```

```
K/if:gfr<30->reject:"contraindicated"
```

```
L/dose=weight_kg*0.8
```

```
D/25@clin=proposal:patient_id:"PX-101"|gfr:28|weight_kg:70|drug:"X"
```

Run (SXR-API TS/Deno):

```
deno run -A tools/sxr/api_ts.ts
```

```
curl -s -X POST :8080/run -H "content-type: application/json" -d @builds/
rx_rule.ir.json
```

Saída:

```
{ "ok": false, "error": "contraindicated" }
```

Demonstra **política clínica** aplicando antes do cálculo, com **determinismo e trilha** (compliance).

5) SLang (procedural) pré-processando antes do SCL

Entrada (SLang, scripts/promo_import.slang):

```
[stm-program promo]
/src:execute = main
μ/discount_pct = 0.10

:func main() {
  sku ← "P-001"
  base_price ← 9.90
  final_price ← base_price * (1 - discount_pct)
  :return { "sku": sku, "price": final_price } # alimenta δ do SCL
}
```

Uso:

```
slangc.py emit-ir scripts/promo_import.slang > builds/promo.ir.json
```

Runner que captura o **result** do SLang e injeta como D/* no **SCL** de pricing (pipeline).

Mostra **integração ETL/transform**: SLang prepara dados, SCL aplica políticas/cálculos.

SXR – Sabores e contratos

A) SXR-Native (tipo “EXE”/one-file)

Input: builds/*.ir.json

Output: dist/<name>/<name>(.exe)

CLI esperado: dist/<name>/<name> [path/to/other.ir.json]

Garantias: determinístico, sem eval, exit code coerente.

B) SXR-JS (Node/Deno/Browser)

Node/Deno CLI: node dist/<name>/svm_bundle.cjs builds/<name>.ir.json

Deno compile: gera binário nativo (svm_node(.exe)).

Browser/PWA: dist/<name>/app.html (seleciona IR, executa offline).

Garantias: mesmo conjunto de opcodes, sem eval livre.

C) SXR-API (serviço HTTP)

POST /run com body = IR JSON.

200 OK: { "ok": true, "result": <any>, "meta": { "engine": "SXR-API", "ir_ver": "svm-ir/1" } }

Erro: { "ok": false, "error": "... " }

Python (Flask/FastAPI) e **TS** (Deno) templates prontos.

Interfaces estáveis

SVM-Python (exec de referência):

```
def run_ir(ir: dict) -> any: ...  
# CLI:  
# python tools/lang/svm.py run builds/<name>.ir.json
```

SVM-TS:

```
export class SVM {  
  constructor(ir: any) {}  
  run(): any {}  
}
```

IR mínimo (exemplo):

```
{  
  "ir": "svm-ir/1",  
  "name": "calc",  
  "entry": "main",  
  "const": {},  
  "funcs": {
```

```
"main": {  
  "params": [],  
  "ops": [  
    ["LOAD_CONST", 2], ["LOAD_CONST", 2], ["BIN_ADD", null],  
    ["RETURN", null]  
  ]  
}
```

Compatibilidade & DoD

Rejeitar IR com "ir" desconhecido (ex.: svm-ir/2) com erro claro.

DoD: Dado um IR válido:

SXR-Native/JS/API retornam **mesma saída** da SVM de referência;

sem eval fora do interpretador de opcodes;

logs de erro incluem **opcode, função, índice**.

Estrutura do repo (mínima)

/tools/lang/

slangc.py # .slang -> IR


```
scl.py      # .scl -> IR
svm.py      # executor referência (run_ir / CLI)
/tools/sxr/
native_pack.py # gera one-file
js_runner.ts  # runner TS (Node/Deno/Browser)
api_py.py    # SXR-API Python
api_ts.ts    # SXR-API TS (Deno)
/contracts/  # *.scl (refund, energy, order, rx)
/scripts/    # *.slang (promo_import, ETL)
/builds/     # *.ir.json (saída de compilação)
/dist/       # artefatos SXR
```

TL;DR para a IA que vai tocar

Implemente **SXR** nas 3 vertentes (Native/JS/API) garantindo que os **4 cases** acima rodem **idênticos** à SVM de referência.

Aceite **apenas** IR com "ir":"svm-ir/1".

Expose run_ir(ir) (Python) e SVM.run() (TS).

Meta: demonstrar RBAC/PII (fintech), $\kappa > \lambda$ (energia), guided slots (e-commerce) e regra clínica (saúde) — sem eval aberto, com auditoria e determinismo.